```
> cd register
[2] pry(#<CashRegister>):1> show-source add_item

From: code/debugging/pry_test.rb:11:
Owner: CashRegister
Visibility: public
Signature: add_item(item, quantity, price_in_cents)
Number of lines: 3

def add_item(item, quantity, price_in_cents)
  @items << {item:, quantity:, price_in_cents:}
end
```

If you type show-source without an argument, it will show the source of the current context's class, or if you are at the top level, the method you were in the middle of when you invoked Pry.

The similar command show-doc will show the documentation of the method or class used as the argument.

Another useful quirk of Pry is that you can easily treat it like a regular Unix command line. Any command you start with a dot (.) is sent to the underlying terminal shell. By using this, you can interact with the operating system directly without using Pry. Pry also overrides the common file reading program cat, you can invoke cat FILENAME without using a ., and Pry will display the output paged and with the syntax highlighted.

Pry is a big program with a plugin-filled ecosystem of its own, and there's more to it that we can cover here. Check out http://pry.github.io for full details.

## Debugging Performance Issues with Benchmark

Ruby is an interpreted, high-level language, and as such it may not perform as fast as a lower-level language such as C. In the following sections, we'll list some basic things you can do to inspect performance.

Typically, slow-running programs have one or two performance graveyards—places where execution time goes to die. Find and improve them, and suddenly your whole program springs back to life. The trick is finding them—developers are notoriously bad at guessing where performance hangups actually are. The Benchmark module can help.

You can use the Benchmark module to time sections of code. For example, we may wonder what the overhead of method invocation is. You can use Benchmark.bm or Benchmark.bmbm to find out.

```
require "benchmark"

LOOP_COUNT = 1_000_000

Benchmark.bmbm(12) do |test|
  test.report("inline:") do
    LOOP_COUNT.times do |x|
      # nothing
    end
  end

  test.report("method:") do
    def method
```